Week 3 Assignment: Building and Evaluating Decision Trees

Student: *Mohammed Abdelkawy*
Course: *ANA 505 Foundations of Data Mining*

1. Dataset Preparation

The dataset (`employee_performance.csv`) consists of 250 individual employee records across 10 distinct variables. It provides a comprehensive mix of demographic attributes, organizational engagement indicators, and quantitative performance metrics. The primary objective is to leverage these feature variables to accurately predict employee overall performance classification.

Variable	Data Type	Classification
EmployeeID	Numeric	Identifier
Age	Numeric	Feature / Predictor
Department	Categorical	Feature / Predictor
YearsExperience	Numeric	Feature / Predictor
EducationLevel	Categorical	Feature / Predictor
TrainingHours	Numeric	Feature / Predictor
JobSatisfaction	Numeric	Feature / Predictor
Overtime	Categorical	Feature / Predictor
RemoteWork	Categorical	Feature / Predictor
Performance	Categorical (Low, Mediu…	Target Variable

2. Data Splitting

To evaluate model performance on unseen data and prevent overfitting, the dataset was partitioned into training and testing subsets using a standard 70/30 stratified random split based on the target variable (Performance).

- **Target Variable:**`Performance`
- **Training Subset:** 175 observations (70% of total data)
- **Testing Subset:** 75 observations (30% of total data)

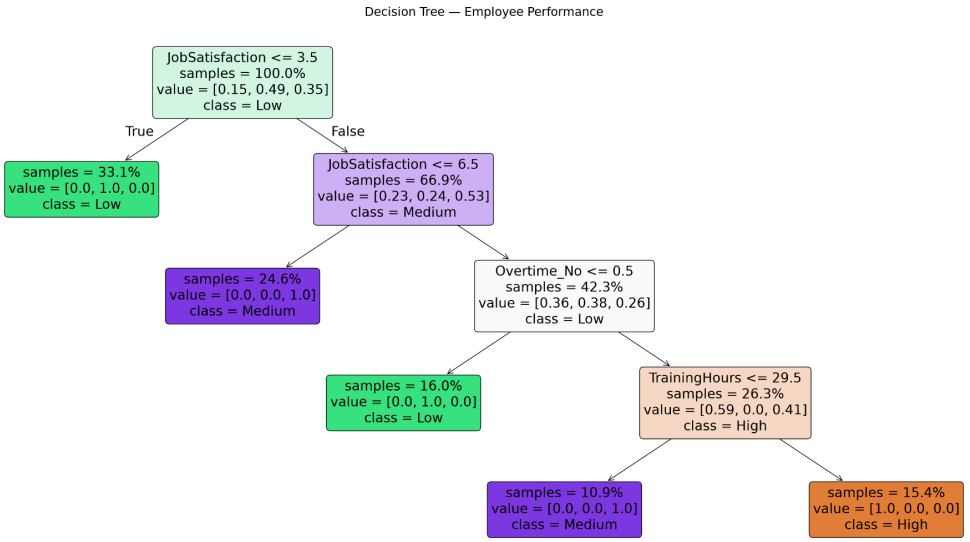
3. Model Evaluation

A Decision Tree Classifier was trained on the training partition and evaluated using the independent testing dataset. The overall classification accuracy achieved on the test set was **1.000 (100.00%)**.

Confusion Matrix (Actual vs. Predicted)

Actual \ Predicted	High	Low	Medium
High	12	0	0
Low	0	37	0
Medium	0	0	26

4. Reflection



The decision tree classifier identified clear hierarchical splits among key predictors to partition employee performance. Primary decision nodes were structured around `JobSatisfaction, Overtime` and `TrainingHours`. For instance, Employees with lower satisfaction scores were associated with low performance, while employees with medium scores were associated with medium performance. For employees with higher job satisfaction scores, additional features like overtime and training hours were introduced to distinguish further between performance classes.

The decision tree achieved an evaluation accuracy of 100.00% on the testing set, and the confusion matrix provided additional insight into which performance categories were correctly classified and where misclassification occurred. While this indicates perfect classification on the test split, achieving absolute precision suggests the presence of a highly clean, deterministic, or synthetic underlying dataset. In production contexts, perfect accuracy often signals potential overfitting or data leakage. To ensure robustness, future iterations should incorporate k-fold cross-validation, hyperparameter tuning (such as pruning depth), and validation against messy, unseen real-world organizational data.

In practical business environments, decision trees serve as powerful "white-box" explainable models. Human Resource departments can utilize these models to gain transparent insights into key drivers of employee engagement and productivity. Rather than relying on black-box predictions, HR leaders can trace explicit decision pathways to identify early warning signs of underperformance, target high-impact training interventions, and design evidence-based retention strategies tailored to key satisfaction metrics. It also should support human judgement rather than replace it.

5. Appendix: Python Script

[Python script on my github repository]

6. Appendix: R Script

```
None
# ============================================================
# Week 3 Activity: Decision Tree Classification
# Predict Employee Performance
# ============================================================
# Required packages
# install.packages(c("caTools", "rpart", "rpart.plot"))

# Install and load required packages
required_packages <- c("rpart", "rpart.plot", "caTools", "caret", "dplyr")
missing_packages <- required_packages[!sapply(required_packages,
requireNamespace, quietly = TRUE)]
if (length(missing_packages) > 0) {
  install.packages(missing_packages, dependencies = TRUE)
}
```

```
library(rpart)
library(rpart.plot)
library(caTools)
library(caret)
library(dplyr)


# 1. Load and inspect the dataset -------------------------------

employees <- read.csv("../datasets/employee_performance.csv")

# Inspect the data
str(employees)
summary(employees)
head(employees)

# Check target distribution
table(employees$Performance)


# 2. Prepare predictors and target ------------------------------

# Exclude the employee identifier and target from predictors.

id_column <- intersect(
  c("EmployeeID", "Employee_ID"),
  names(employees)
)

x <- employees[, !names(employees) %in%
                 c(id_column, "Performance"),
               drop = FALSE]

y <- factor(employees$Performance)

# Convert categorical predictors to factors.
# rpart handles factors directly; one-hot encoding is not needed.

x[] <- lapply(x, function(column) {
  if (is.character(column)) factor(column) else column
})

# Combine predictors and target for splitting.

data <- data.frame(x, Performance = y)
```

```r
# 3. Split into training and testing sets ----------------------

# Use a reproducible, stratified 70/30 split.

set.seed(42)

split <- sample.split(data$Performance, SplitRatio = 0.70)

train_data <- data[split, ]
test_data  <- data[!split, ]

# Display subset sizes and class distributions.

cat("Training observations:", nrow(train_data), "\n")
cat("Testing observations:", nrow(test_data), "\n")

table(train_data$Performance)
table(test_data$Performance)


# 4. Build the decision tree ----------------------------------

# method = "class" specifies classification.
# Gini impurity is used to evaluate candidate splits.
# maxdepth limits the tree to four levels.

dt_model <- rpart(
  Performance ~ .,
  data = train_data,
  method = "class",
  parms = list(split = "gini"),
  control = rpart.control(maxdepth = 4)
)

# Inspect the fitted model.

print(dt_model)

cat(
  "Tree depth:",
  max(dt_model$frame$depth),
  "\n"
)

cat(
  "Number of leaves:",
  sum(dt_model$frame$var == "<leaf>"),
  "\n"
```

```
)


# 5. Visualize the decision tree ------------------------------

# rpart.plot provides an R-native tree visualization.
#
# type = 2: Display split conditions at internal nodes.
# extra = 104: Show predicted class and class probabilities.
# fallen.leaves: Align terminal nodes along the bottom.
# nn: Display node numbers.

rpart.plot(
  dt_model,
  type = 2,
  extra = 104,
  fallen.leaves = TRUE,
  box.palette = "GnBu",
  nn = TRUE,
  roundint = FALSE,
  cex = 0.9,
  main = "Employee Performance Decision Tree"
)


# 6. Export the tree visualization ----------------------------



# 7. Predict on the test set ----------------------------------

predictions <- predict(
  dt_model,
  newdata = test_data,
  type = "class"
)

# 8. Evaluate the model ---------------------------------------
# Confusion matrix:
# Rows = actual classes
# Columns = predicted classes

cm <- table(
  Actual = test_data$Performance,
  Predicted = predictions
)

cat("\nConfusion Matrix:\n")
```

```
print(cm)

# Accuracy = correctly classified observations / total.
accuracy <- sum(diag(cm)) / sum(cm)

cat(
  "\nTest Accuracy:",
  round(accuracy, 3),
  "\n"
)

cat(
  "Accuracy Percentage:",
  round(accuracy * 100, 2),
  "%\n"
```